

You can order print and ebook versions of *Think Bayes 2e* from Bookshop.org and [Amazon](http://Amazon.com).

Comparison

```
# install empiricaldist if necessary
```

```
try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
    import empiricaldist
```

```
# Get utils.py
```

```
from os.path import basename, exists
```

```
def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve
        local, _ = urlretrieve(url, filename)
        print('Downloaded ' + local)
```

```
download('https://github.com/AllenDowney/ThinkBayes2/raw/master/soln/utils.py')
```

```
from utils import set_pyplot_params
set_pyplot_params()
```

This chapter introduces joint distributions, which are an essential tool for working with distributions of more than one variable.

We'll use them to solve a silly problem on our way to solving a real problem. The silly problem is figuring out how tall two people are, given only that one is taller than the other. The real problem is rating chess players (or participants in other kinds of competition) based on the outcome of a game.

To construct joint distributions and compute likelihoods for these problems, we will use outer products and similar operations. And that's where we'll start.

Outer Operations

Many useful operations can be expressed as the "outer product" of two sequences, or another kind of "outer" operation. Suppose you have sequences like `x` and `y`:

```
x = [1, 3, 5]
y = [2, 4]
```

The outer product of these sequences is an array that contains the product of every pair of values, one from each sequence. There are several ways to compute outer products, but the one I think is the most versatile is a "mesh grid".

NumPy provides a function called `meshgrid` that computes a mesh grid. If we give it two sequences, it returns two arrays.

```
import numpy as np

X, Y = np.meshgrid(x, y)
```

The first array contains copies of `x` arranged in rows, where the number of rows is the length of `y`.

X

```
array([[1, 3, 5],
       [1, 3, 5]])
```

The second array contains copies of `y` arranged in columns, where the number of columns is the length of `x`.

Y

```
array([[2, 2, 2],
       [4, 4, 4]])
```

Because the two arrays are the same size, we can use them as operands for arithmetic functions like multiplication.

X * Y

```
array([[ 2,  6, 10],
       [ 4, 12, 20]])
```

This result is the outer product of `x` and `y`. We can see that more clearly if we put it in a `DataFrame`:

```
import pandas as pd

df = pd.DataFrame(X * Y, columns=x, index=y)
df
```

	1	3	5
2	2	6	10
4	4	12	20

The values from `x` appear as column names; the values from `y` appear as row labels. Each element is the product of a value from `x` and a value from `y`.

We can use mesh grids to compute other operations, like the outer sum, which is an array that contains the *sum* of elements from `x` and elements from `y`.

```
X + Y
```

```
array([[3, 5, 7],
       [5, 7, 9]])
```

We can also use comparison operators to compare elements from `x` with elements from `y`.

```
X > Y
```

```
array([[False,  True,  True],
       [False, False,  True]])
```

The result is an array of Boolean values.

It might not be obvious yet why these operations are useful, but we'll see examples soon. With that, we are ready to take on a new Bayesian problem.

How Tall Is A?

Suppose I choose two people from the population of adult males in the U.S.; I'll call them A and B. If we see that A taller than B, how tall is A?

To answer this question:

1. I'll use background information about the height of men in the U.S. to form a prior distribution of height,
2. I'll construct a joint prior distribution of height for A and B (and I'll explain what that is),
3. Then I'll update the prior with the information that A is taller, and
4. From the joint posterior distribution I'll extract the posterior distribution of height for A.

In the U.S. the average height of male adults is 178 cm and the standard deviation is 7.7 cm. The distribution is not exactly normal, because nothing in the real world is, but the normal distribution is a pretty good model of the actual distribution, so we can use it as a prior distribution for A and B.

Here's an array of equally-spaced values from 3 standard deviations below the mean to 3 standard deviations above (rounded up a little).

```
mean = 178
qs = np.arange(mean-24, mean+24, 0.5)
```

SciPy provides a function called `norm` that represents a normal distribution with a given mean and standard deviation, and provides `pdf`, which evaluates the probability density function (PDF) of the normal distribution:

```
from scipy.stats import norm

std = 7.7
ps = norm(mean, std).pdf(qs)
```

Probability densities are not probabilities, but if we put them in a `Pmf` and normalize it, the result is a discrete approximation of the normal distribution.

```
from empiricaldist import Pmf

prior = Pmf(ps, qs)
prior.normalize()
```

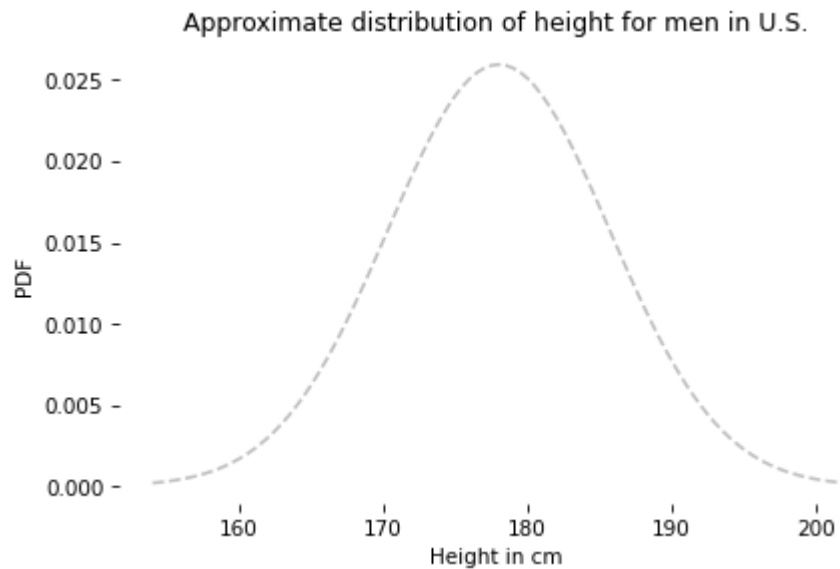
```
np.float64(1.9963309462450582)
```

Here's what it looks like.

```
from utils import decorate

prior.plot(style='--', color='C5')

decorate(xlabel='Height in cm',
         ylabel='PDF',
         title='Approximate distribution of height for men in U.S.')
```



This distribution represents what we believe about the heights of **A** and **B** before we take into account the data that **A** is taller.

Joint Distribution

The next step is to construct a distribution that represents the probability of every pair of heights, which is called a joint distribution. The elements of the joint distribution are

$$P(A_x \text{ and } B_y)$$

which is the probability that `A` is x cm tall and `B` is y cm tall, for all values of x and y .

At this point all we know about `A` and `B` is that they are male residents of the U.S., so their heights are independent; that is, knowing the height of `A` provides no additional information about the height of `B`.

In that case, we can compute the joint probabilities like this:

$$P(A_x \text{ and } B_y) = P(A_x) P(B_y)$$

Each joint probability is the product of one element from the distribution of `x` and one element from the distribution of `y`.

So if we have `Pmf` objects that represent the distribution of height for `A` and `B`, we can compute the joint distribution by computing the outer product of the probabilities in each `Pmf`.

The following function takes two `Pmf` objects and returns a `DataFrame` that represents the joint distribution.

```
def make_joint(pmf1, pmf2):
    """Compute the outer product of two Pmfs."""
    X, Y = np.meshgrid(pmf1, pmf2)
    return pd.DataFrame(X * Y, columns=pmf1.qs, index=pmf2.qs)
```

The column names in the result are the quantities from `pmf1`; the row labels are the quantities from `pmf2`.

In this example, the prior distributions for `A` and `B` are the same, so we can compute the joint prior distribution like this:

```
joint = make_joint(prior, prior)
joint.shape
```

```
(96, 96)
```

The result is a `DataFrame` with possible heights of `A` along the columns, heights of `B` along the rows, and the joint probabilities as elements.

If the prior is normalized, the joint prior is also be normalized.

```
joint.to_numpy().sum()
```

```
np.float64(1.0)
```

To add up all of the elements, we convert the `DataFrame` to a NumPy array before calling `sum`. Otherwise, `DataFrame.sum` would compute the sums of the columns and return a `Series`.

```
series = joint.sum()  
series.shape
```

```
(96,)
```

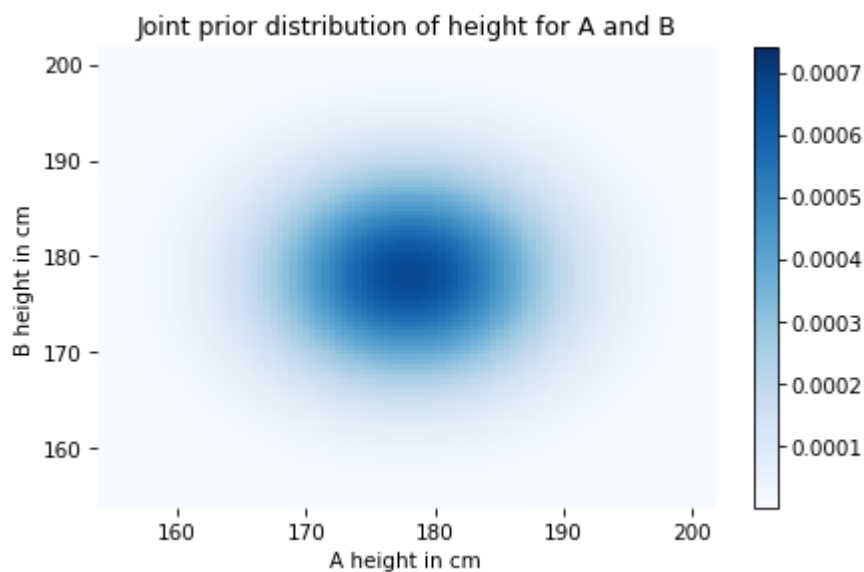
Visualizing the Joint Distribution

The following function uses `pcolormesh` to plot the joint distribution.

```
import matplotlib.pyplot as plt  
  
def plot_joint(joint, cmap='Blues'):  
    """Plot a joint distribution with a color mesh."""  
    vmax = joint.to_numpy().max() * 1.1  
    plt.pcolormesh(joint.columns, joint.index, joint,  
                  cmap=cmap,  
                  vmax=vmax,  
                  shading='nearest')  
    plt.colorbar()  
  
    decorate(xlabel='A height in cm',  
            ylabel='B height in cm')
```

Here's what the joint prior distribution looks like.

```
plot_joint(joint)  
decorate(title='Joint prior distribution of height for A and B')
```

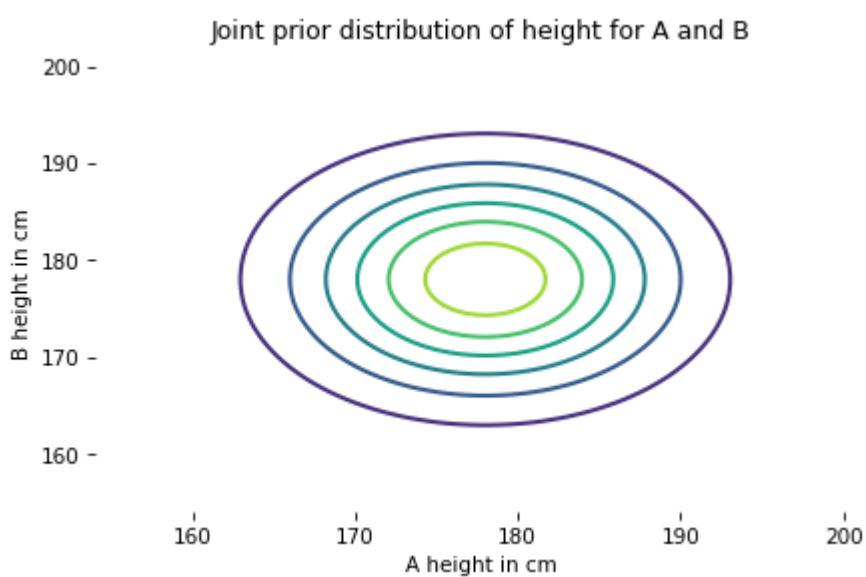


As you might expect, the probability is highest (darkest) near the mean and drops off farther from the mean.

Another way to visualize the joint distribution is a contour plot.

```
def plot_contour(joint):  
    """Plot a joint distribution with a contour."""  
    plt.contour(joint.columns, joint.index, joint,  
                linewidths=2)  
    decorate(xlabel='A height in cm',  
            ylabel='B height in cm')
```

```
plot_contour(joint)  
decorate(title='Joint prior distribution of height for A and B')
```



Each line represents a level of equal probability.

Likelihood

Now that we have a joint prior distribution, we can update it with the data, which is that `A` is taller than `B`.

Each element in the joint distribution represents a hypothesis about the heights of `A` and `B`. To compute the likelihood of every pair of quantities, we can extract the column names and row labels from the prior, like this:

```
x = joint.columns
y = joint.index
```

And use them to compute a mesh grid.

```
X, Y = np.meshgrid(x, y)
```

`X` contains copies of the quantities in `x`, which are possible heights for `A`. `Y` contains copies of the quantities in `y`, which are possible heights for `B`.

If we compare `X` and `Y`, the result is a Boolean array:

```
A_taller = (X > Y)
A_taller.dtype
```

```
dtype('bool')
```

To compute likelihoods, I'll use `np.where` to make an array with `1` where `A_taller` is `True` and `0` elsewhere.

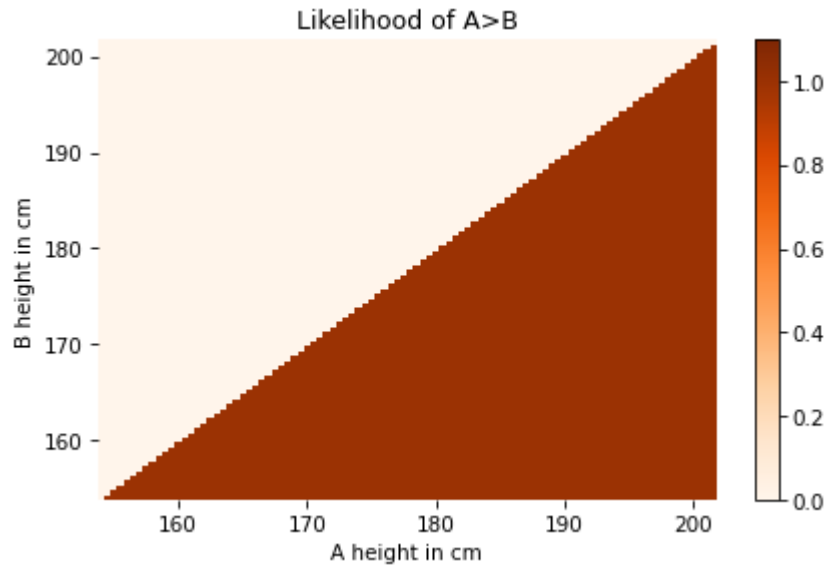
```
a = np.where(A_taller, 1, 0)
```

To visualize this array of likelihoods, I'll put in a `DataFrame` with the values of `x` as column names and the values of `y` as row labels.

```
likelihood = pd.DataFrame(a, index=x, columns=y)
```

Here's what it looks like:

```
plot_joint(likelihood, cmap='Oranges')
decorate(title='Likelihood of A>B')
```



The likelihood of the data is 1 where $X > Y$ and 0 elsewhere.

The Update

We have a prior, we have a likelihood, and we are ready for the update. As usual, the unnormalized posterior is the product of the prior and the likelihood.

```
posterior = joint * likelihood
```

I'll use the following function to normalize the posterior:

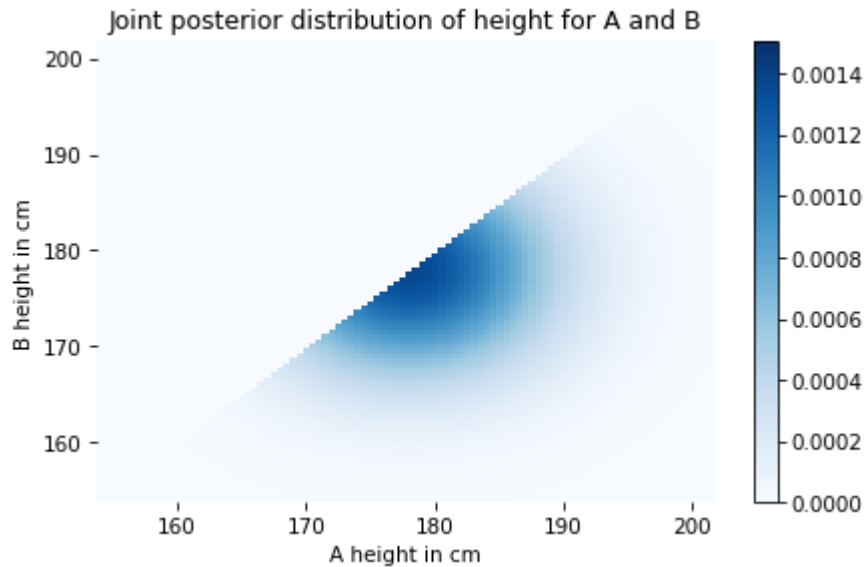
```
def normalize(joint):
    """Normalize a joint distribution."""
    prob_data = joint.to_numpy().sum()
    joint /= prob_data
    return prob_data
```

```
normalize(posterior)
```

```
np.float64(0.49080747821526977)
```

And here's what it looks like.

```
plot_joint(posterior)
decorate(title='Joint posterior distribution of height for A and B')
```



All pairs where **B** is taller than **A** have been eliminated. The rest of the posterior looks the same as the prior, except that it has been renormalized.

Marginal Distributions

The joint posterior distribution represents what we believe about the heights of **A** and **B** given the prior distributions and the information that **A** is taller.

From this joint distribution, we can compute the posterior distributions for **A** and **B**. To see how, let's start with a simpler problem.

Suppose we want to know the probability that **A** is 180 cm tall. We can select the column from the joint distribution where `x=180`.

```
column = posterior[180]
column.head()
```

```
154.0    0.000010
154.5    0.000013
155.0    0.000015
155.5    0.000019
156.0    0.000022
Name: 180.0, dtype: float64
```

This column contains posterior probabilities for all cases where `x=180`; if we add them up, we get the total probability that **A** is 180 cm tall.

```
column.sum()
```

```
np.float64(0.03017221271570807)
```

It's about 3%.

Now, to get the posterior distribution of height for `A`, we can add up all of the columns, like this:

```
column_sums = posterior.sum(axis=0)
column_sums.head()
```

```
154.0    0.000000e+00
154.5    1.012260e-07
155.0    2.736152e-07
155.5    5.532519e-07
156.0    9.915650e-07
dtype: float64
```

The argument `axis=0` means we want to add up the columns.

The result is a `Series` that contains every possible height for `A` and its probability. In other words, it is the distribution of heights for `A`.

We can put it in a `Pmf` like this:

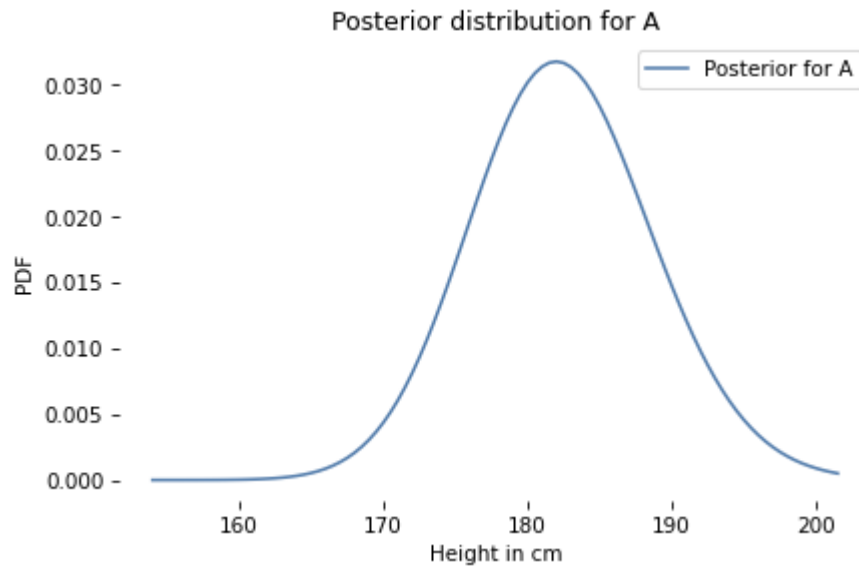
```
marginal_A = Pmf(column_sums)
```

When we extract the distribution of a single variable from a joint distribution, the result is called a **marginal distribution**. The name comes from a common visualization that shows the joint distribution in the middle and the marginal distributions in the margins.

Here's what the marginal distribution for `A` looks like:

```
marginal_A.plot(label='Posterior for A')

decorate(xlabel='Height in cm',
         ylabel='PDF',
         title='Posterior distribution for A')
```



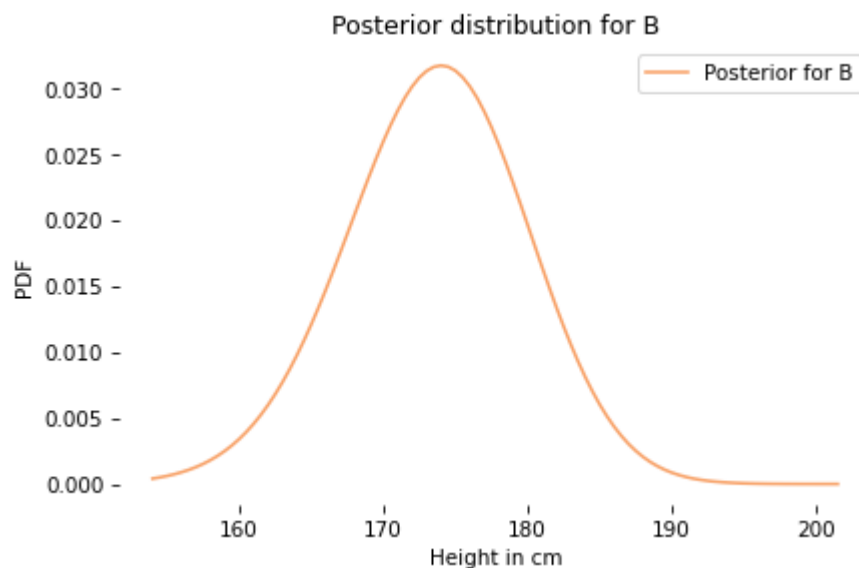
Similarly, we can get the posterior distribution of height for **B** by adding up the rows and putting the result in a `Pmf`.

```
row_sums = posterior.sum(axis=1)
marginal_B = Pmf(row_sums)
```

Here's what it looks like.

```
marginal_B.plot(label='Posterior for B', color='C1')

decorate(xlabel='Height in cm',
         ylabel='PDF',
         title='Posterior distribution for B')
```



Let's put the code from this section in a function:

```
def marginal(joint, axis):
    """Compute a marginal distribution."""
    return Pmf(joint.sum(axis=axis))
```

`marginal` takes as parameters a joint distribution and an axis number:

- If `axis=0`, it returns the marginal of the first variable (the one on the x-axis);
- If `axis=1`, it returns the marginal of the second variable (the one on the y-axis).

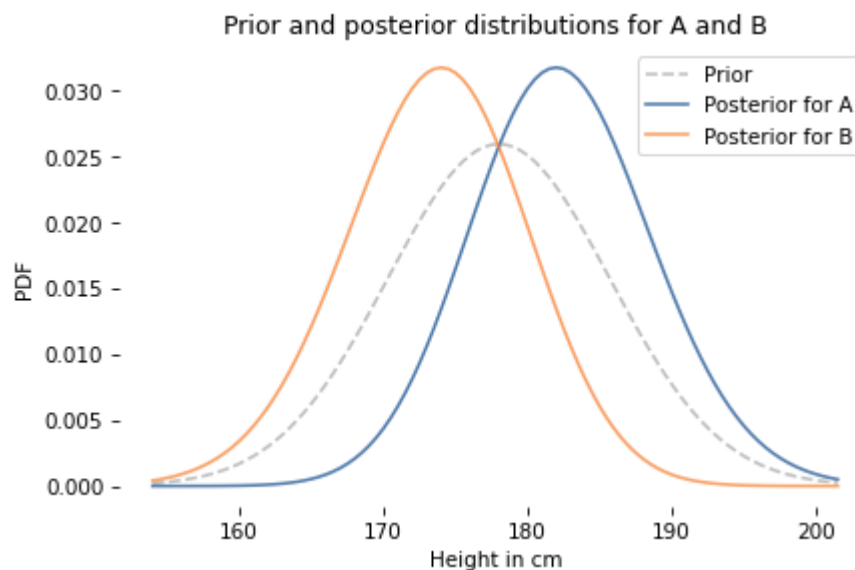
So we can compute both marginals like this:

```
marginal_A = marginal(posterior, axis=0)
marginal_B = marginal(posterior, axis=1)
```

Here's what they look like, along with the prior.

```
prior.plot(style='--', label='Prior', color='C5')
marginal_A.plot(label='Posterior for A')
marginal_B.plot(label='Posterior for B')

decorate(xlabel='Height in cm',
          ylabel='PDF',
          title='Prior and posterior distributions for A and B')
```



As you might expect, the posterior distribution for `A` is shifted to the right and the posterior distribution for `B` is shifted to the left.

We can summarize the results by computing the posterior means:

```
prior.mean()
```

```
np.float64(177.99516026921506)
```

```
print(marginal_A.mean(), marginal_B.mean())
```

```
182.38728123421686 173.60286000233387
```

Based on the observation that **A** is taller than **B**, we are inclined to believe that **A** is a little taller than average, and **B** is a little shorter.

Notice that the posterior distributions are a little narrower than the prior. We can quantify that by computing their standard deviations.

```
prior.std()
```

```
np.float64(7.624924796641578)
```

```
print(marginal_A.std(), marginal_B.std())
```

```
6.27046117764547 6.280513548175111
```

The standard deviations of the posterior distributions are a little smaller, which means we are more certain about the heights of **A** and **B** after we compare them.

Conditional Posteriors

Now suppose we measure **A** and find that he is 170 cm tall. What does that tell us about **B**?

In the joint distribution, each column corresponds a possible height for **A**. We can select the column that corresponds to height 170 cm like this:

```
column_170 = posterior[170]
```

The result is a **Series** that represents possible heights for **B** and their relative likelihoods. These likelihoods are not normalized, but we can normalize them like this:

```
cond_B = Pmf(column_170)
cond_B.normalize()
```

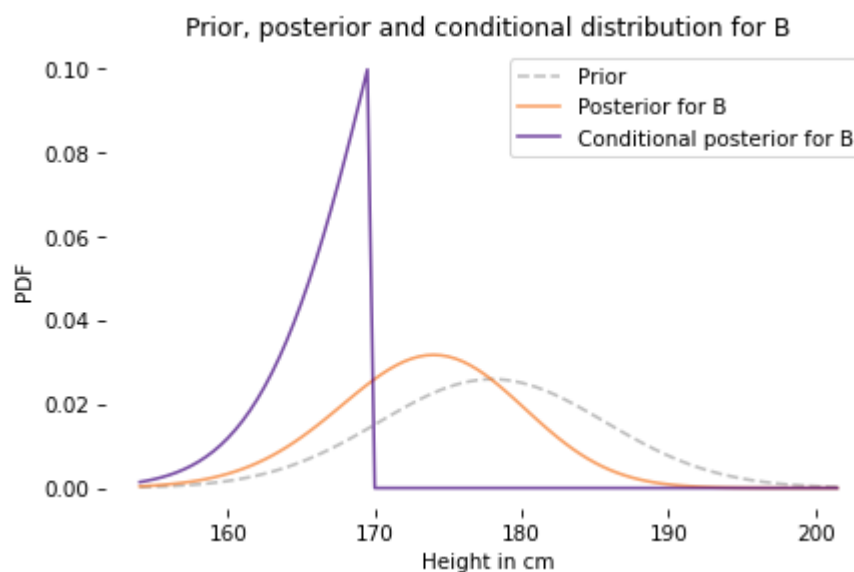
```
np.float64(0.004358061205454471)
```

Making a `Pmf` copies the data by default, so we can normalize `cond_B` without affecting `column_170` or `posterior`. The result is the conditional distribution of height for `B` given that `A` is 170 cm tall.

Here's what it looks like:

```
prior.plot(style='--', label='Prior', color='C5')
marginal_B.plot(label='Posterior for B', color='C1')
cond_B.plot(label='Conditional posterior for B',
            color='C4')

decorate(xlabel='Height in cm',
        ylabel='PDF',
        title='Prior, posterior and conditional distribution for B')
```



The conditional posterior distribution is cut off at 170 cm, because we have established that `B` is shorter than `A`, and `A` is 170 cm.

Dependence and Independence

When we constructed the joint prior distribution, I said that the heights of `A` and `B` were independent, which means that knowing one of them provides no information about the other. In other words, the conditional probability $P(A_x|B_y)$ is the same as the unconditional probability $P(A_x)$.

But in the posterior distribution, `A` and `B` are not independent. If we know that `A` is taller than `B`, and we know how tall `A` is, that gives us information about `B`.

The conditional distribution we just computed demonstrates this dependence.

Summary

In this chapter we started with the "outer" operations, like outer product, which we used to construct a joint distribution.

In general, you cannot construct a joint distribution from two marginal distributions, but in the special case where the distributions are independent, you can.

We extended the Bayesian update process and applied it to a joint distribution. Then from the posterior joint distribution we extracted marginal posterior distributions and conditional posterior distributions.

As an exercise, you'll have a chance to apply the same process to a problem that's a little more difficult and a lot more useful, updating a chess player's rating based on the outcome of a game.

Exercises

Exercise: Based on the results of the previous example, compute the posterior conditional distribution for `A` given that `B` is 180 cm.

Hint: Use `loc` to select a row from a `DataFrame`.

```
# Solution

# Select a row from the posterior and normalize it

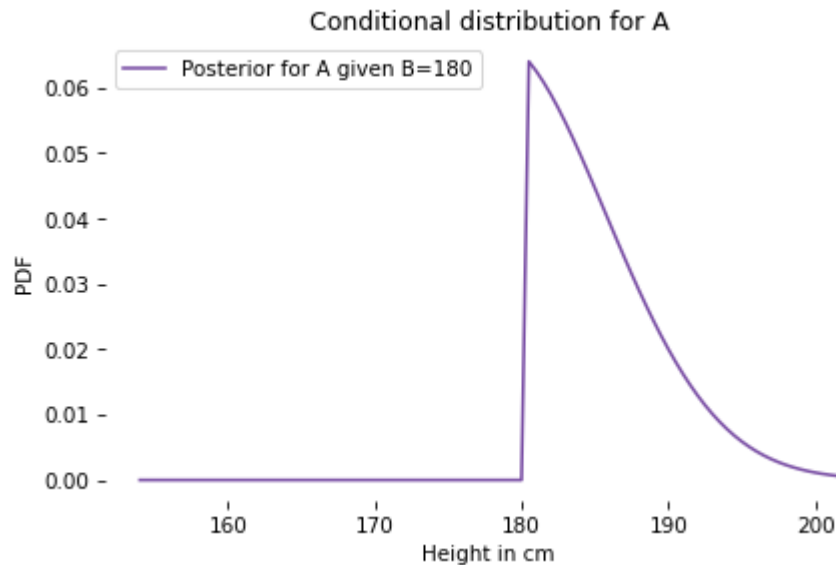
row_180 = posterior.loc[180]
cond_A = Pmf(row_180)
cond_A.normalize()
```

```
np.float64(0.019669089649708035)
```

```
# Solution

# Here's what it looks like

cond_A.plot(label='Posterior for A given B=180', color='C4')
decorate(xlabel='Height in cm',
         ylabel='PDF',
         title='Conditional distribution for A')
```



Exercise: Suppose we have established that **A** is taller than **B**, but we don't know how tall **B** is. Now we choose a random woman, **C**, and find that she is shorter than **A** by at least 15 cm. Compute posterior distributions for the heights of **A** and **C**.

The average height for women in the U.S. is 163 cm; the standard deviation is 7.3 cm.

```
# Solution

# Here's a prior distribution for the height of
# a randomly chosen woman

mean = 163
qs = np.arange(mean-24, mean+24, 0.5)

std = 7.3
ps = norm(mean, std).pdf(qs)

prior_C = Pmf(ps, qs)
prior_C.normalize()
```

```
np.float64(1.997970387889823)
```

```
# Solution

# Here's the joint prior for A and C

joint_AC = make_joint(marginal_A, prior_C)
joint_AC.shape
```

```
(96, 96)
```

```
# Solution

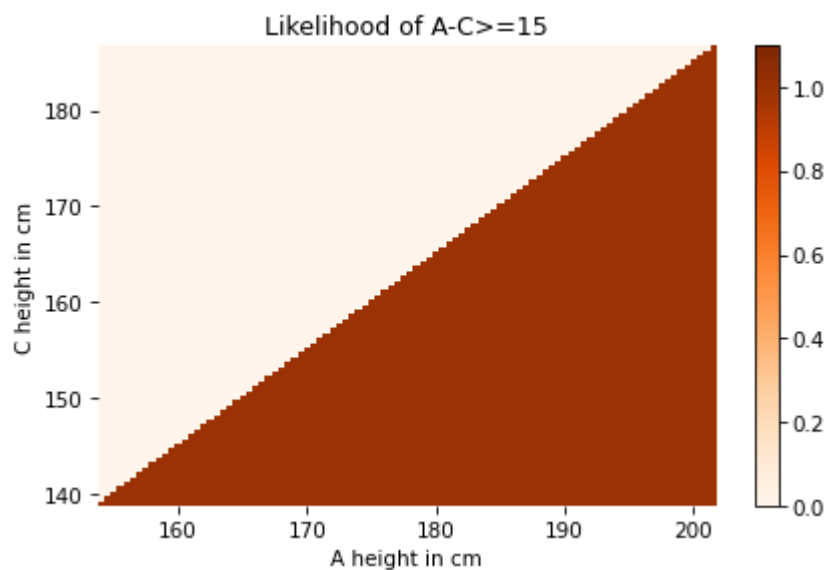
# To compute the likelihood of the data, we'll
# use a meshgrid

x = joint_AC.columns
y = joint_AC.index
X, Y = np.meshgrid(x, y)
a = np.where(X-Y>=15, 1, 0)
likelihood_AC = pd.DataFrame(a, index=y, columns=x)
```

```
# Solution

# Here's what the likelihood looks like

plot_joint(likelihood_AC, cmap='Oranges')
decorate(ylabel='C height in cm',
         title='Likelihood of A-C>=15')
```



```
# Solution

# Here's the update

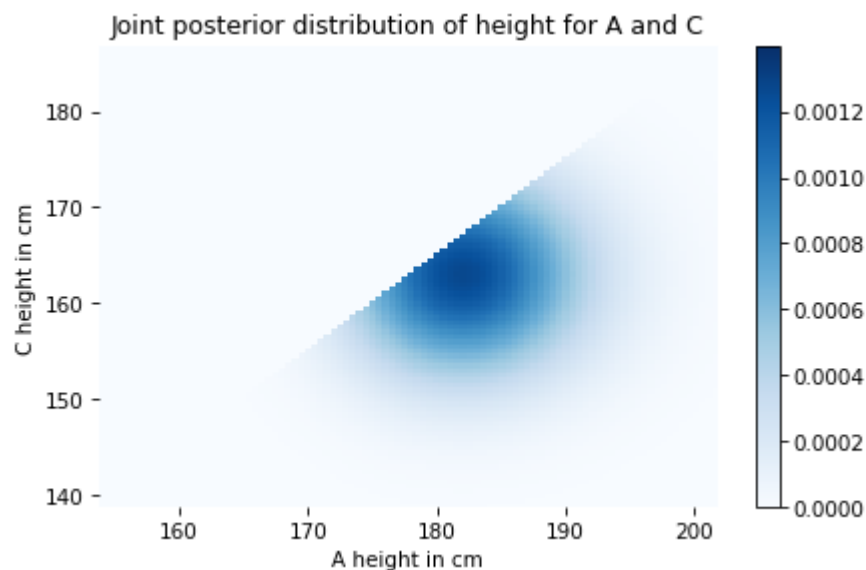
posterior_AC = joint_AC * likelihood_AC
normalize(posterior_AC)
```

```
np.float64(0.6839061829242193)
```

```
# Solution

# And the joint posterior

plot_joint(posterior_AC)
decorate(ylabel='C height in cm',
         title='Joint posterior distribution of height for A and C')
```



```
# Solution

# Here are the marginal posterior distributions

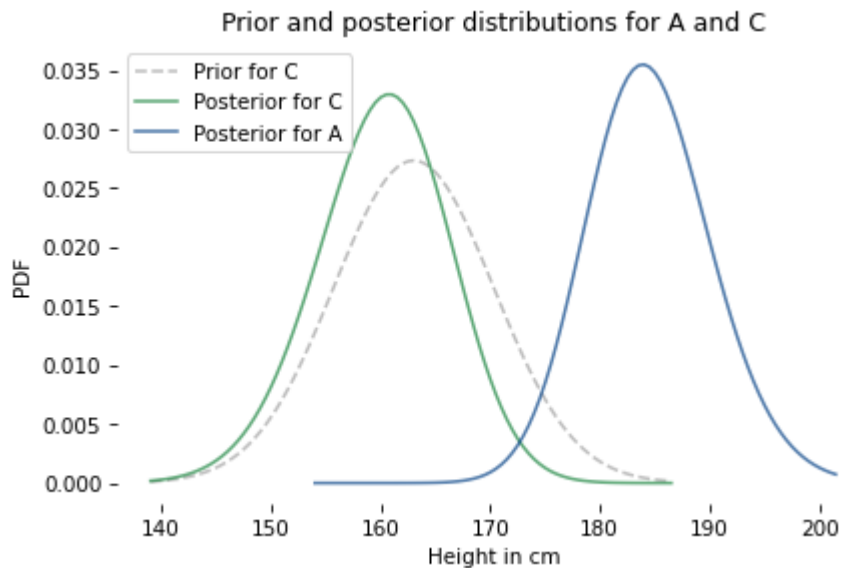
marginal_AC = marginal(posterior_AC, axis=0)
marginal_C = marginal(posterior_AC, axis=1)
```

```
# Solution

# And here's what they look like

prior_C.plot(style='--', label='Prior for C', color='C5')
marginal_C.plot(label='Posterior for C', color='C2')
marginal_AC.plot(label='Posterior for A', color='C0')

decorate(xlabel='Height in cm',
         ylabel='PDF',
         title='Prior and posterior distributions for A and C')
```



Exercise: The Elo rating system is a way to quantify the skill level of players for games like chess.

It is based on a model of the relationship between the ratings of players and the outcome of a game. Specifically, if R_A is the rating of player A and R_B is the rating of player B, the probability that A beats B is given by the logistic function:

$$P(\text{A beats B}) = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

The parameters 10 and 400 are arbitrary choices that determine the range of the ratings. In chess, the range is from 100 to 2800.

Notice that the probability of winning depends only on the difference in rankings. As an example, if R_A exceeds R_B by 100 points, the probability that A wins is

```
1 / (1 + 10**(-100/400))
```

```
0.6400649998028851
```

Suppose A has a current rating of 1600, but we are not sure it is accurate. We could describe their true rating with a normal distribution with mean 1600 and standard deviation 100, to indicate our uncertainty.

And suppose B has a current rating of 1800, with the same level of uncertainty.

Then A and B play and A wins. How should we update their ratings?

To answer this question:

1. Construct prior distributions for **A** and **B**.
2. Use them to construct a joint distribution, assuming that the prior distributions are independent.
3. Use the logistic function above to compute the likelihood of the outcome under each joint hypothesis.
4. Use the joint prior and likelihood to compute the joint posterior.
5. Extract and plot the marginal posteriors for **A** and **B**.
6. Compute the posterior means for **A** and **B**. How much should their ratings change based on this outcome?

Solution

Here are the priors for A and B

```
qs = np.arange(1300, 1900, 10)
ps = norm(1600, 100).pdf(qs)
prior_A_elo = Pmf(ps, qs)
prior_A_elo.normalize()
```

```
qs = np.arange(1500, 2100, 10)
ps = norm(1800, 100).pdf(qs)
prior_B_elo = Pmf(ps, qs)
prior_B_elo.normalize()
```

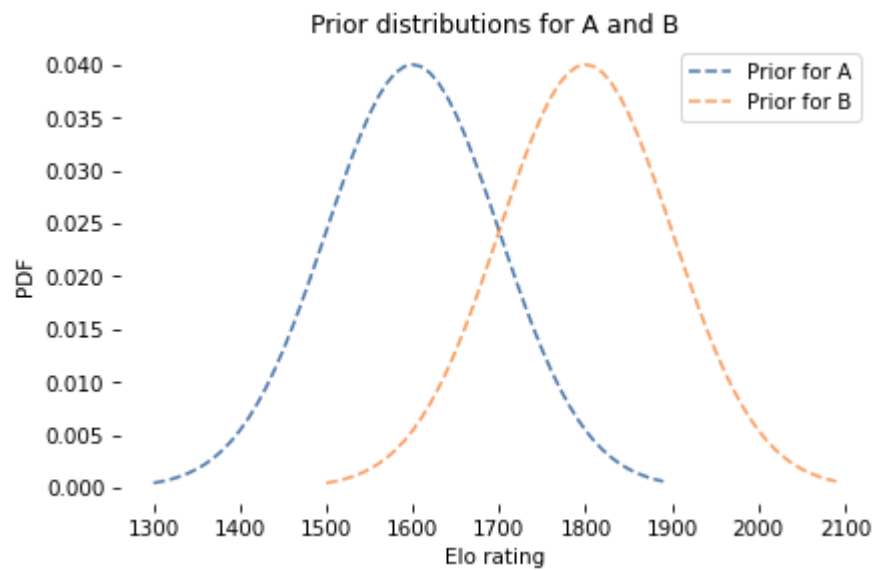
```
np.float64(0.09972780668486173)
```

Solution

Here's what the priors look like

```
prior_A_elo.plot(style='--', label='Prior for A')
prior_B_elo.plot(style='--', label='Prior for B')

decorate(xlabel='Elo rating',
         ylabel='PDF',
         title='Prior distributions for A and B')
```



```
# Solution
```

```
# Here is the joint prior distribution
```

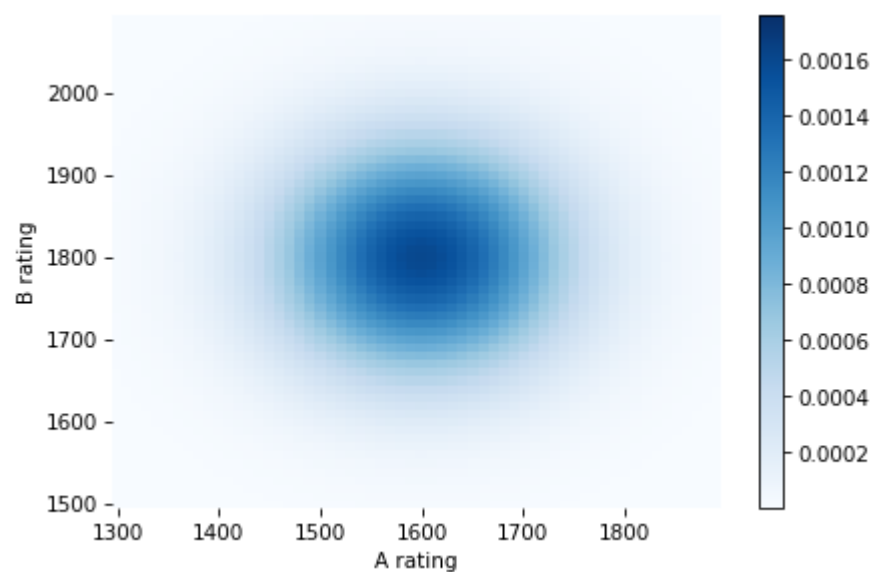
```
joint_elo = make_joint(prior_A_elo, prior_B_elo)  
joint_elo.shape
```

```
(60, 60)
```

```
# Solution
```

```
# And here's what it looks like
```

```
plot_joint(joint_elo)  
decorate(xlabel='A rating',  
         ylabel='B rating')
```



```
# Solution
```

```
# Here's a meshgrid we can use to compute differences in rank
```

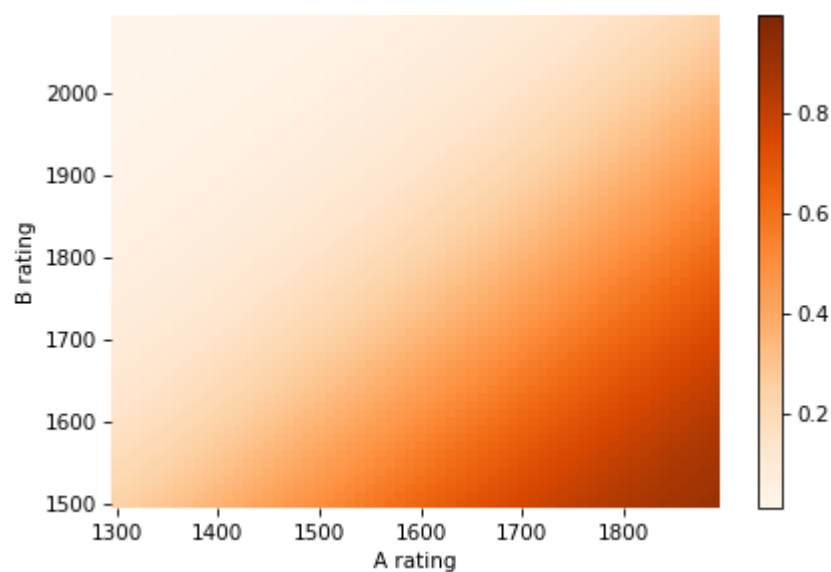
```
x = joint_elo.columns
y = joint_elo.index
X, Y = np.meshgrid(x, y)
diff = X - Y
```

```
# Solution
```

```
# And here are the likelihoods
```

```
a = 1 / (1 + 10**(-diff/400))
likelihood_elo = pd.DataFrame(a, columns=x, index=y)

plot_joint(likelihood_elo, cmap='Oranges')
decorate(xlabel='A rating',
         ylabel='B rating')
```



```
# Solution
```

```
# Here's the update
```

```
posterior_elo = joint_elo * likelihood_elo
normalize(posterior_elo)
```

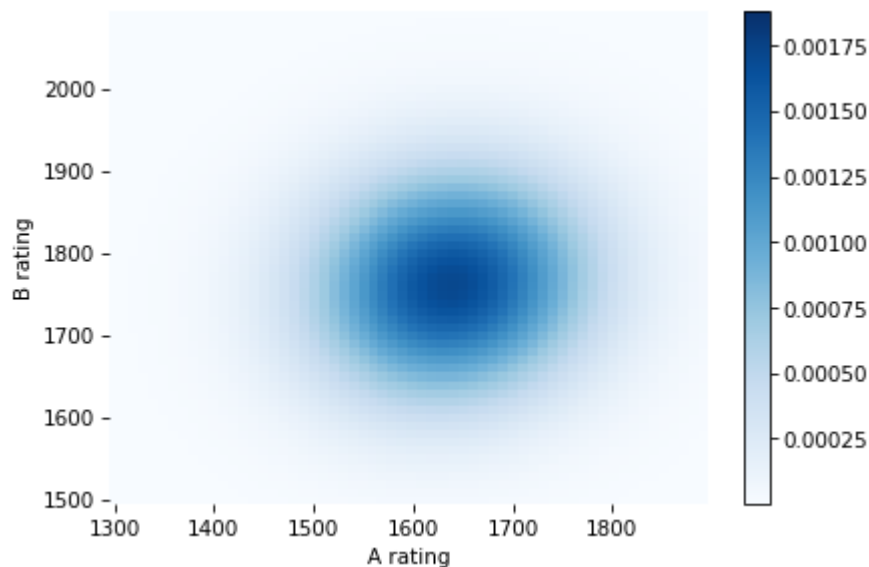
```
np.float64(0.2660426288107942)
```



```
# Solution
```

```
# Here's what the joint posterior looks like
```

```
plot_joint(posterior_elo)
decorate(xlabel='A rating',
         ylabel='B rating')
```



```
# Solution
```

```
# Here are the marginal posterior distributions
```

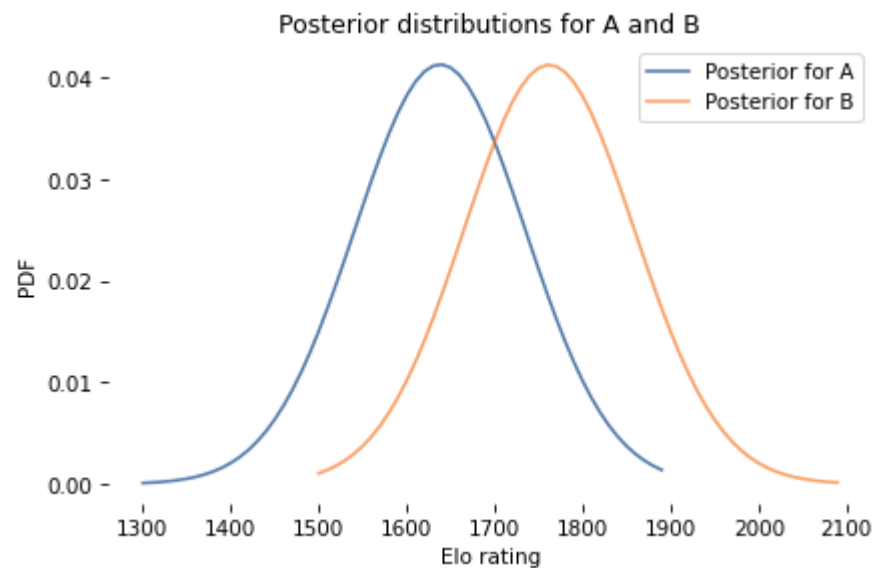
```
marginal_A_elo = marginal(posterior_elo, axis=0)
marginal_B_elo = marginal(posterior_elo, axis=1)
```

```
# Solution
```

```
# Here's what they look like
```

```
marginal_A_elo.plot(label='Posterior for A')
marginal_B_elo.plot(label='Posterior for B')

decorate(xlabel='Elo rating',
         ylabel='PDF',
         title='Posterior distributions for A and B')
```



```
# Solution

# Posterior means

marginal_A_elo.mean(), marginal_B_elo.mean()
```

```
(np.float64(1636.648345528236), np.float64(1763.0203078793095))
```

```
# Solution

# Posterior standard deviation

marginal_A_elo.std(), marginal_B_elo.std()
```

```
(np.float64(95.34063582447712), np.float64(95.61569535990881))
```

Think Bayes, Second Edition

Copyright 2020 Allen B. Downey

License: Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)